Advanced Data Grid Blueprint

A Comprehensive Guide & Specification for the Internship Portfolio Project

1. Project Overview

This project is a high-performance, interactive Data Grid designed to simulate an admin dashboard. It demonstrates the ability to manage complex state, handle asynchronous data fetching, and manipulate large arrays dynamically in React. It is specifically architected to prove production-ready skills to tech recruiters.

- **Tech Stack:** React (UI), Tailwind CSS (Styling), Zustand (State Management), Vite (Bundler).
- **Core Objective:** Build a scalable data table featuring multi-criteria filtering, debounced searching, dynamic sorting, and client-side pagination.
- **Data Source:** FakeStore API (or similar mock JSON provider).

2. Component Architecture

The application is modularized to ensure separation of concerns. The shell handles the layout, while the data grid components handle the logic and presentation of the data.

```
📦 src/
├ 📂 components/
│ ├ 📂 layout/
│ │ ├ 📜 DashboardLayout.jsx  (Master wrapper)
│ │ ├ 📜 Sidebar.jsx          (Static navigation)
│ │ └ 📜 TopNav.jsx           (Header & user profile)
│ └ 📂 grid/
│   ├ 📜 DataGrid.jsx        (Parent container, reads state)
│   ├ 📜 FilterBar.jsx       (Search input & dropdowns)
│   ├ 📜 Table.jsx           (HTML table structure)
│   ├ 📜 TableHeader.jsx     (Sortable column headers)
│   ├ 📜 TableRow.jsx        (Maps specific data fields)
│   └ 📜 Pagination.jsx      (Next/Prev controls)
├ 📂 store/
│ └ 📜 useGridStore.js     (Zustand global state)
└ 📜 App.jsx
```

3. Zustand State Management Schema

Zustand acts as the single source of truth for the grid. By maintaining this globally, the user's view (filters, current page) remains intact even if they navigate away to a hypothetical "Product Details" page and return.

```
import { create } from 'zustand';

export const useGridStore = create((set) => ({
  // 1. Server Data
  data: [],
  isLoading: false,
  error: null,

  // 2. Active Filters
  searchQuery: "",
  selectedCategory: "All",
  sortConfig: { key: "id", direction: "asc" },

  // 3. Pagination Limits
  currentPage: 1,
  itemsPerPage: 10,

  // 4. Actions (Setters)
  setSearchQuery: (query) => set({ searchQuery: query, currentPage: 1 }),
  setSelectedCategory: (category) => set({ selectedCategory: category, currentPage:
  setSortConfig: (key) => set((state) => {
    // Logic to toggle asc/desc if the same key is clicked
    const isAsc = state.sortConfig.key === key && state.sortConfig.direction === "a
    return { sortConfig: { key, direction: isAsc ? "desc" : "asc" } };
  }),
  setPage: (pageNumber) => set({ currentPage: pageNumber }),

  // 5. Async Fetch Action
  fetchData: async () => {
    set({ isLoading: true });
    try {
      const response = await fetch('https://fakestoreapi.com/products');
      const result = await response.json();
      set({ data: result, isLoading: false });
    } catch (error) {
      set({ error: error.message, isLoading: false });
    }
  }
}));
```

4. The Logic Pipeline: Deriving Data

You must not directly mutate your raw `data` array. Instead, you create a "Pipeline" in your `DataGrid.jsx` component that processes the raw data sequentially before passing it to the Table. This requires strong JavaScript array methods.

> **The Processing Sequence:**
>
> 1. **Filter by Category:** `data.filter(item => item.category === selectedCategory)`
> 2. **Filter by Search:** `...filter(item => item.title.toLowerCase().includes(searchQuery))`
> 3. **Sort Data:** `...sort((a, b) => compare(a[sortKey], b[sortKey]))`
> 4. **Paginate Data:** `...slice(startIndex, endIndex)`

Only the final, chunked array (e.g., exactly 10 items) gets mapped to `<TableRow />` components.

5. The 15-Day Execution Roadmap

This plan distributes the workload evenly, moving from infrastructure to presentation to complex logic.

Timeline	Goals & Tasks	Core Focus
Days 1 - 3: Setup	• Initialize Vite + React project. • Install Tailwind CSS and Zustand. • Create `useGridStore.js` and write the `fetchData` async function. • Render raw JSON onto the screen to verify connection.	Infrastructure & API handling
Days 4 - 6: Layout	• Build the `DashboardLayout`, `Sidebar`, and Top Navigation. • Build the raw HTML `<table>` structure. • Apply Tailwind CSS to make the table look professional and handle horizontal scrolling.	Component structure & CSS/UI Polish

Timeline	Goals & Tasks	Core Focus
Days 7 - 9: Filters	• Create `FilterBar.jsx` (Search input + Category dropdown). • Write the array filtering logic in the parent component. • Ensure changing a filter resets the `currentPage` back to 1.	Array `.filter()` methods & State updates
Days 10 - 12: Data Control	• Add `onClick` events to Table headers to trigger sorting. • Implement array `.sort()` logic (handle both strings and numbers). • Calculate pagination slices and build the `Pagination` component.	Algorithms (Sorting/ Math for limits)
Days 13 - 15: Final Polish	• Add a Debounce hook to the search bar to prevent lag. • Add empty states ("No results found") and loading spinners. • Deploy to Vercel/Netlify. • Write a comprehensive GitHub README.	UX improvements & Deployment

6. Essential Edge Cases to Handle

To truly impress recruiters, your logic must handle unexpected scenarios smoothly. Ensure you account for the following during development:

- **Empty Search Results:** If a user searches for "xyza", the table should not break. It should display a clean "No matching products found" UI within the table body.
- **Pagination Boundaries:** If there are 25 items and itemsPerPage is 10, the last page should only display 5 items. The "Next" button must be disabled on page 3.

- **Case-Insensitive Searching:** Always convert both the user's query and the data field to lowercase before comparing them.
- **Debouncing:** Do not update the Zustand store on every single keystroke. Wait for the user to stop typing for ~300ms before triggering the search filter to save memory.